

Términos para defender de memoria

Lo justo para improvisar tranquilo. Cada término: **qué es** en una línea · **cómo aparece en el TP** · **qué decir si preguntan**.

Solo lenguaje de clase. Los términos con nombre de autor (Poisson, Markov, Kendall-Lee, Little, Erlang) van **descritos**, no dados por sabidos. Unidades: **I fundamentos** · **II aleatoriedad** · **III colas**.

UNIDAD I · Fundamentos: sistema, modelo, simulación

Sistema

Conjunto de entidades que interactúan con un objetivo común.

En el TP: el sistema real es "defender una base de oleadas".

Entidad

Objeto de interés del sistema. **En el TP:** un enemigo, una torre.

Atributo

Propiedad de una entidad. **En el TP:** la temperatura de una torre.

Estado del sistema

Las variables que describen al sistema en un instante. **En el TP:** longitud de la cola + temperatura de cada torre.

Evento

Un cambio de estado. **En el TP:** llega un enemigo, una torre termina de disparar, una torre se sobrecalienta.

Modelo

Representación simplificada de un sistema real que conserva solo lo relevante para la pregunta que querés responder.

En el TP: el modelo es una red de líneas de espera (colas); la pregunta es "¿cuántas torres conviene?".

Si preguntan por tipos de modelo: el nuestro es **matemático, dinámico, estocástico y de eventos discretos**. (Las otras opciones serían: físico, estático, determinístico, continuo.)

Simulación

Experimentar sobre el **modelo** (no sobre el sistema real) para estimar cómo se comporta.

¿Cuándo se simula? Cuando el sistema es estocástico/complejo, no hay fórmula cerrada exacta, o experimentar en la realidad es caro o riesgoso.

Determinístico vs. Estocástico

Determinístico: misma entrada → misma salida, sin azar. **Estocástico:** hay variables aleatorias. **En el TP:** estocástico — llegadas y ataques son aleatorios.

Estático vs. Dinámico

Estático: sin tiempo. **Dinámico:** evoluciona en el tiempo. **En el TP:** dinámico.

Verificación vs. Validación (V&V)

Verificar = “¿programé bien el modelo?” (el código hace lo que el modelo dice). **Validar** = “¿el modelo representa la realidad?”.

En el TP: validamos comparando la simulación contra la teoría de colas (caso sin temperatura) y con la Ley de Little.

Etapas de un estudio de simulación

Formular el problema → datos → modelo conceptual → programa → **verificación** → **validación** → diseño de experimentos → corridas → análisis → documentar.

En el TP: es exactamente el orden de las 4 Waves de la charla.

UNIDAD II · Aleatoriedad: números pseudoaleatorios y variables aleatorias

Número pseudoaleatorio (PRNG)

Número que **parece** aleatorio pero lo genera una fórmula determinística a partir de una semilla. La base de toda la aleatoriedad de la simulación.

Generadores del apunte: de **cuadrados medios** y **congruencial** (lineal y multiplicativo). Detalle de implementación: Python usa internamente otro generador por defecto, pero eso no es tema del programa — si surge, aclaralo así.

Semilla (seed) y reproducibilidad

El valor inicial del generador. Misma semilla ⇒ misma secuencia de números ⇒ misma corrida.

En el TP: usamos **semilla 42**; por eso el output .json es reproducible y auditable: cualquiera repite la misma batalla.

Variable aleatoria (v.a.)

Cantidad cuyo valor depende del azar, descrita por una distribución de probabilidad.

En el TP: el tiempo entre llegadas y el tiempo de servicio son variables aleatorias.

Transformada inversa

Método para generar una variable aleatoria con la distribución que querés, a partir de un número uniforme entre 0 y 1 ($R \sim U(0,1)$): se invierte su función de distribución acumulada.

Si preguntan: es cómo convertimos los números pseudoaleatorios “crudos” en tiempos exponenciales para las llegadas y los servicios.

Proceso de Poisson ↔ Distribución exponencial

Si los eventos (llegadas) ocurren a tasa constante λ de forma independiente —eso se llama **proceso de Poisson**, por el matemático Poisson—, entonces los **tiempos entre eventos** siguen una **distribución exponencial** de parámetro λ . Son dos caras de lo mismo.

En el TP: llegadas Poisson de tasa λ ; servicios exponenciales de tasa μ .

Markoviano / "sin memoria"

Una distribución es "sin memoria" cuando lo que falta para el próximo evento no depende de cuánto ya esperaste. La exponencial tiene esa propiedad; por eso a esos modelos se los llama **markovianos** (la "M" de la notación), por el matemático Markov.

Clave del TP: la **temperatura** de las torres **rompe** esta propiedad (el servicio pasa a "tener memoria") → por eso no alcanza la fórmula y hay que **simular**.

UNIDAD III · Teoría de colas (líneas de espera)

Sistema de líneas de espera (cola)

Clientes que llegan, esperan si el servicio está ocupado, son atendidos y se van. Sus **3 piezas**: proceso de **llegadas**, mecanismo de **servicio**, y **disciplina** de la cola.

En el TP: enemigo = cliente · torre = servidor · esperar torre libre = cola · fugarse por lleno = bloqueo.

Notación de Kendall-Lee

Formato estándar para describir una cola —lleva el nombre de los autores Kendall y Lee—: (A / B / c) (disciplina / K / N). A = distribución de llegadas, B = de servicio, c = servidores, K = capacidad del sistema, N = tamaño de la población.

El nuestro: (M/M/c) (FIFO/K/∞) → llegadas y servicio markovianos, c torres, orden de llegada, capacidad finita K, población infinita.

FIFO (disciplina)

"First In, First Out": se atiende por **orden de llegada**. **En el TP:** la cola normal. (La alternativa que probamos es **prioridad**.)

Población infinita (∞)

La tasa de llegada **no depende** de cuántos ya hay dentro. **En el TP:** asumimos ∞.

Capacidad finita K · Bloqueo / pérdida

El sistema admite a lo sumo **K** clientes entre cola y servicio. El que llega y lo encuentra lleno **se pierde** (bloqueo). Hace que el modelo sea "con pérdida".

En el TP: el bloqueo es la fuga del enemigo a la base. **K=10**.

Factor de utilización ρ (rho)

$\rho = \lambda / (c \cdot \mu)$ — fracción del tiempo que, en promedio, cada servidor está ocupado.

En el TP: $\lambda=0,40 \cdot \mu=0,25 \cdot c=3 \rightarrow \rho = 0,533$. Es **cuenta directa**, no simulación.

Estabilidad: con cola infinita se necesita $\rho < 1$, si no la fila crece sin límite. Con K finita nunca "explota", pero aparece el bloqueo.

Familias M/M/1 · M/M/c · M/M/c/K

M/M/1: un servidor. **M/M/c:** c servidores en paralelo. **M/M/c/K:** c servidores con capacidad limitada K (con bloqueo).

En el TP: el nuestro es **M/M/c/K**; comparamos contra **M/M/c** teórico para validar.

Medidas de desempeño: L , W , L_q , W_q

L = nº medio de clientes en el sistema · W = tiempo medio en el sistema · L_q = nº medio en cola · W_q = tiempo medio de espera en cola. (la "q" es de **queue**, cola.)

Ley de Little

Relación de la teoría de colas —lleva el nombre del autor Little—: el nº medio de clientes en el sistema es igual a la tasa de llegada por el tiempo medio de permanencia: $L = \lambda \cdot W$.

En el TP: es una de las pruebas de validación — chequeamos que nuestros números la cumplan.

Erlang B / Erlang C

Fórmulas de la teoría de colas —del ingeniero Erlang—: **Erlang B** da la probabilidad de **bloqueo** (sistema lleno); **Erlang C** da la probabilidad de **tener que esperar**.

Estado estacionario

Cuando el sistema ya "se asentó" y sus promedios no dependen del arranque. Las fórmulas de colas valen en estacionario.

Simulación por eventos discretos

El reloj **salta de evento a evento** (no avanza segundo a segundo): arribo, fin de servicio, sobrecalentamiento. De esos saltos sale la línea de tiempo.

En el TP: es lo que hace SimPy en el Paso 1; el resultado se guarda en `output.json`.

Glosario exprés (una línea cada uno)

λ (lambda)	tasa de llegadas — enemigos por segundo
μ (mu)	tasa de servicio por torre — kills por segundo
c	número de torres = servidores
K	capacidad del sistema (cola + servicio)
ρ (rho)	utilización = $\lambda/(c \cdot \mu)$; estable si $\rho < 1$
$a = \lambda/\mu$	intensidad ofrecida (en Erlangs)
FIFO	se atiende por orden de llegada
Bloqueo	el que llega con el sistema lleno se va (= la fuga)
Little	$L = \lambda \cdot W$ (cantidad media = tasa × tiempo medio)
Estacionario	el sistema ya se asentó
Markoviano	"sin memoria"; lo que rompe la temperatura
Semilla	misma semilla → misma corrida (reproducible)

OJO — términos que NO conviene usar (no están en el programa; si te los nombran, contestá con la idea, no con el nombre): Pollaczek-Khinchine / M-G-1 · regla $c\mu$ · "conservación del trabajo" · lote económico / EOQ. Y **Mersenne Twister** es solo el generador interno de Python, **no** tema de clase: mencionalo como detalle de implementación si hace falta.